How Portfolio Workbench works

Deep analysis generated 2026-09-07 · app booted via npm run dev; screenshots are real captures · model claude-sonnet-5

Overview

Portfolio Workbench is Avishek Chandra's personal project portfolio, built as a "Git-backed" website where every project is a plain Markdown file with YAML front matter rather than a row in a database. The Astro site (astro.config.mjs, src/content.config.ts) reads a `projects` content collection from `src/content/projects/<id>-<slug>/index.md`, validates it against a Zod schema (`src/lib/project-schema.ts`), and renders an index page (a carousel of project cards) plus one detail page per project. The philosophy, stated on the homepage footer, is "Markdown is the record. Git is the history. This interface is the view."

The same codebase runs in two distinct modes, selected by the `PUBLIC_ONLY` environment variable. In **local workbench mode** (`npm run dev`, `PUBLIC_ONLY` unset), the app is a full local editor: a development-only Vite middleware (`src/lib/local-editor.ts`) exposes REST-like endpoints bound to the loopback interface that write directly to the project's Markdown/artifact files on disk. Because private projects are sensitive, editing them locally requires a WebAuthn "device unlock" (Windows Hello / fingerprint / PIN) implemented in `src/lib/device-auth.ts` and invoked from the browser via `src/lib/device-unlock-client.ts`. In **hosted mode** (`PUBLIC_ONLY=1`, deployed as a static site to GitHub Pages, `avisheksportfolio.com`), there is no server: instead, editing works by committing directly to the GitHub Contents API using a fine-grained Personal Access Token the owner pastes once into a "connections" dialog in the header. All writes — field edits, artifact uploads, deletes, project creation — become individual Git commits, and GitHub Pages rebuilds the site roughly a minute later.

Private projects are handled specially: their real content lives in a separate, non-public repository (`workbench-private`), which the public build never touches. At deploy time, `scripts/generate-private-stubs.mjs` fetches metadata from the vault (id, status, dates) and writes a "locked stub" project into the public content collection with generic name/description so the project card renders honestly without leaking sensitive text. When a visitor opens a locked project's detail page and authenticates with GitHub, `PrivateVault.astro` fetches the real record and its artifacts client-side straight from the vault repo and lets the owner edit it exactly like a public project — except every save commits to the vault.

The workbench also has an AI layer. "Draft from repository" (`src/lib/ai-complete.ts`, `src/lib/ai/*`) reads the linked GitHub repo/PR through the GitHub API (evidence collection is deterministic, budgeted, and redacts secrets), builds a bounded evidence package, and asks an LLM (Anthropic directly, or OpenAI through a bundled Cloudflare Worker CORS proxy) to draft the project's description and why-it-was-built text. Pressing Draft also commits an `analysis-requests/<folder>.json` file, which a GitHub Actions workflow (`deep-analysis.yml` + `scripts/deep-analysis.mjs`) picks up in the background: it clones the linked repo, boots the app (via docker compose/Dockerfile/npm/python heuristics), crawls it with Playwright, screenshots every page, has the model document every control, typesets a how-it-works PDF, and commits everything into the project's artifacts — even if the browser tab that requested it is long closed. The site polls the request file so any visit shows a run still in flight.

Architecture

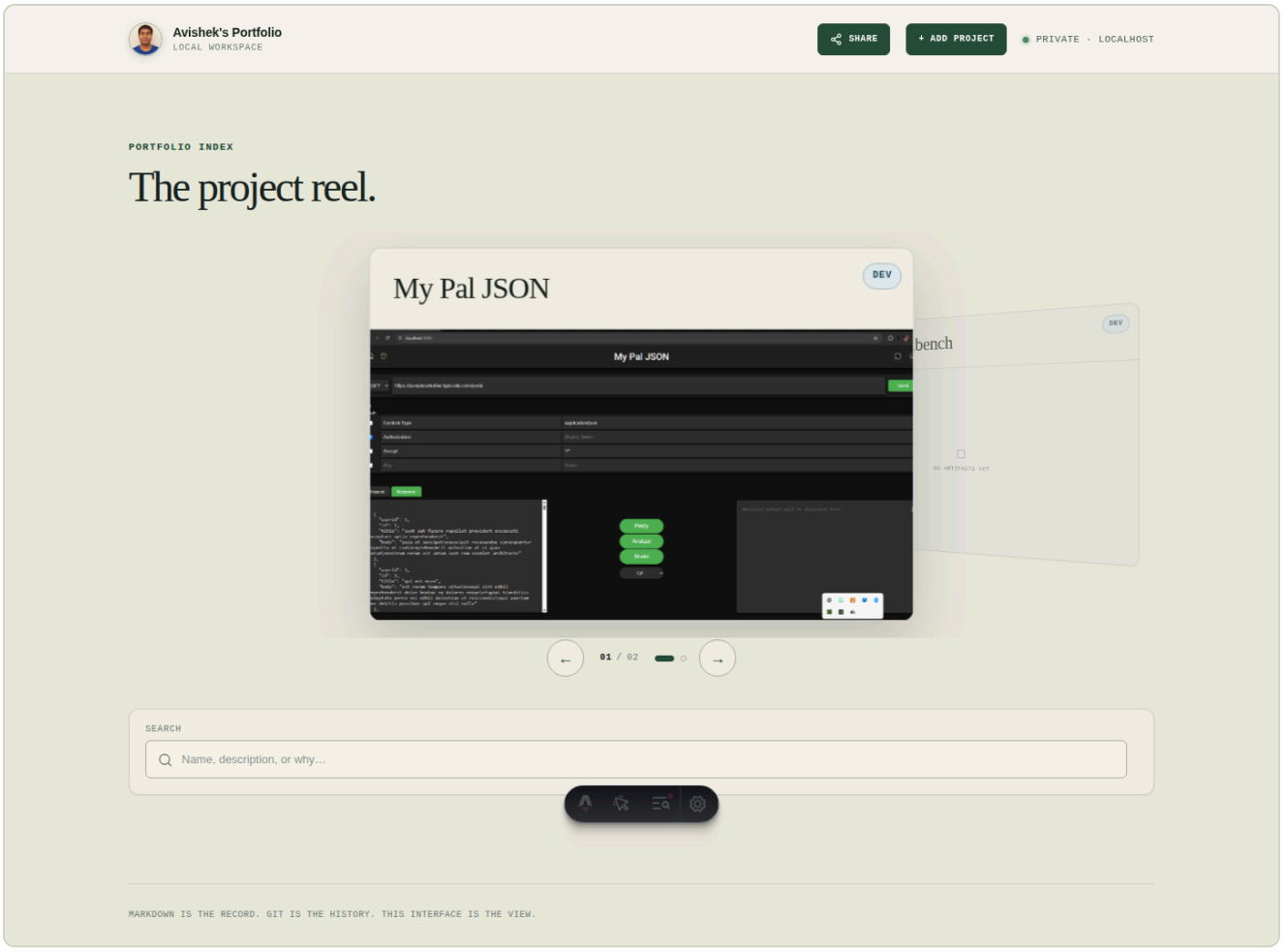
The stack is Astro 7 (static output) + TypeScript, styled with a single global stylesheet, using Astro's content collections with a Zod schema (`project-schema.ts`) as the source of truth for a project record's shape (`id`, `slug`, `name`, `description`, `why`, `status`, `privacy`, `startDate`/`updatedDate`, optional `artifactOrder`, `featuredArtifact`, `repositoryUrl`, `pullRequestUrl`, `deleted`). Each project folder also has an `artifacts/` directory holding arbitrary files (images, video, PDF, or "file" fallback), served through a static endpoint (`src/pages/project-artifacts/[project]/[...file].ts`) that is generated per-artifact at build time via `getStaticPaths`. A shared carousel module (`src/lib/artifact-carousel.ts`) generates identical HTML markup whether rendered server-side (public pages) or injected client-side after unlock (private vault), so behavior is consistent everywhere.

Editing logic is split cleanly by environment. `src/lib/local-editor.ts` is a Vite plugin registered in `astro.config.mjs` that intercepts dev-server requests, enforces loopback-only access, requires WebAuthn device-unlock for private-project writes, validates payloads against the Zod schemas, and writes Markdown/artifact files straight to `src/content/projects/`. `src/lib/github-client.ts` is the hosted-mode equivalent: a thin wrapper over the GitHub REST Contents API (get/put/delete file, list folders) used for both the public repo (`aviman1258/project-workbench`) and the private vault repo (`aviman1258/workbench-private`), authenticated with a token kept in `localStorage`. `src/lib/project-editor.ts` and `src/lib/field-editor.ts` sit above both backends as shared UI/logic: they define the editable field specs, open themed `<dialog>` editors, patch YAML front matter with an optimistic-concurrency retry on 409, and wire the "Draft from repo" flow and its background-analysis polling (`src/lib/ai/analysis-status.ts`, `draft-progress.ts`).

Data flow for a page view: Astro's `getStaticPaths` in `src/pages/projects/[slug].astro` enumerates all "alive" (non-deleted) projects from the content collection, and for each renders a detail page with the metadata, a carousel of artifacts (loaded from disk via `src/lib/projects.ts`), and either a `RemoteEditor` (hosted, public project), a `PrivateVault` (hosted, private project — locked stub with client-side unlock), or an inline local editor block (dev mode). Data flow for a write: a browser control (pencil icon, add-media button, delete pill) opens a modal or triggers a fetch, which — depending on mode — either POSTs to the local Vite middleware (writing files on disk) or calls the GitHub Contents API (creating a commit), after which the UI updates the DOM in place and/or expects the next Pages rebuild to reflect the change site-wide. GitHub Actions (`deploy.yml`, `convert-pptx.yml`, `deep-analysis.yml`) glue the repo to the live site: every push to `main` rebuilds and republishes; artifact PowerPoint uploads get converted to PDF; and analysis-request commits trigger the deep-analysis crawl-and-document pipeline.

Page: /

The portfolio's home page — a searchable, carousel-style index of all public (and, in local dev, private) projects, sorted by most recently updated.



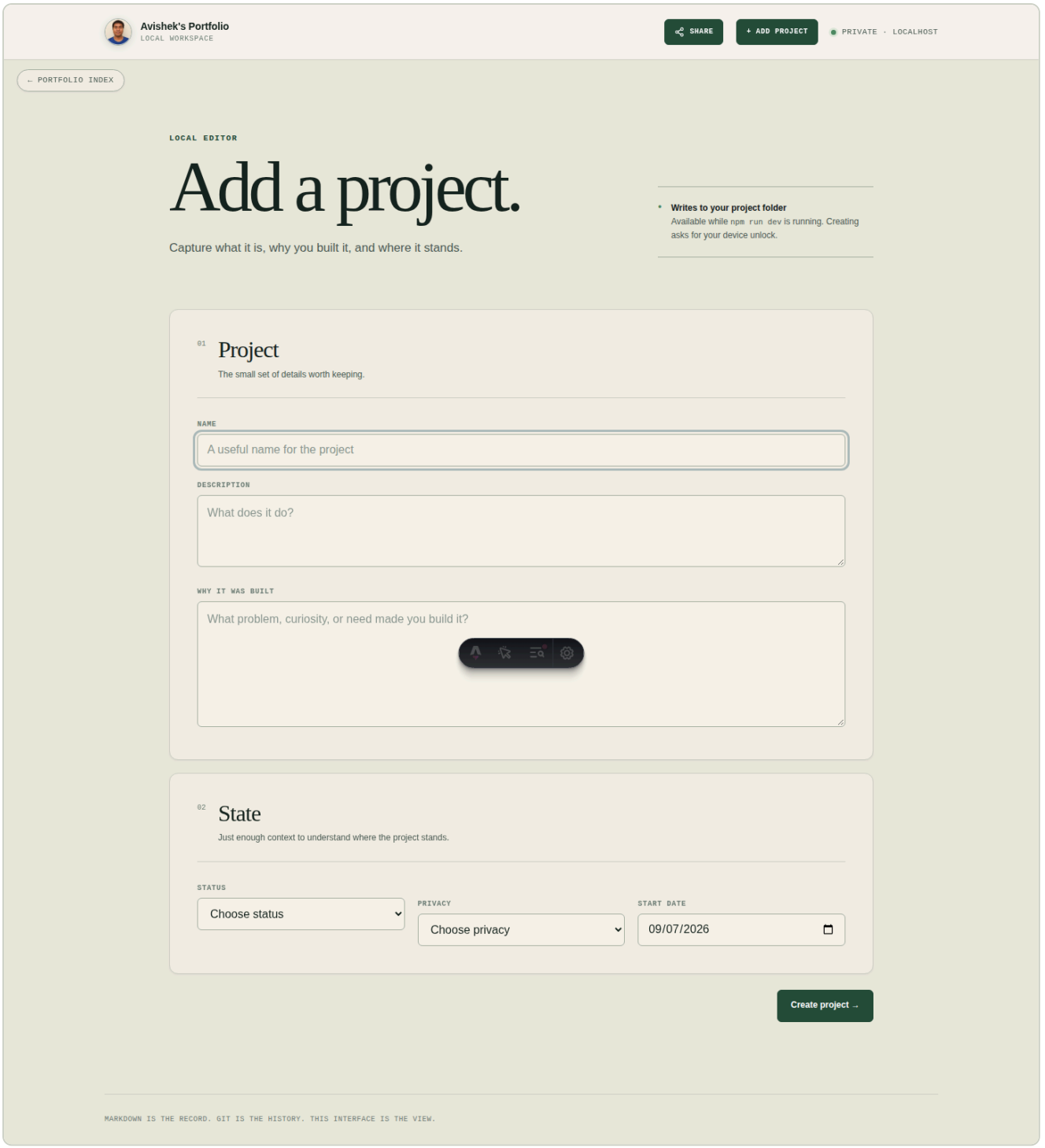
Controls

CONTROL	WHAT IT DOES
Contact card for Avishek Chandra (button, header)	Opens a business-card dialog (defined in `BaseLayout.astro`, `data-card-open` handler) showing name, email, phone, and certification links.
Share (button, header)	Opens the share dialog (`data-share-open` in `BaseLayout.astro`) to copy/share the current page URL.
+ Add project (a, header)	Navigates to `/manage` (`withBase('/manage')` link in `BaseLayout.astro`).
My Pal JSON / Portfolio	Link to `/projects/my-pal-json` and `/projects/porfolio-workbench` respectively, generated by `ProjectCard.astro`'s `<a href={withBase('/projects/${data.slug}')}>`.

Workbench (a, project cards)	
□ **No artifacts yet (a)**	Empty-artifact fallback link on a project card without any artifacts, navigating to that project's detail page (`ProjectCard.astro`).
Show previous project / Show next project (button)	Step the coverflow carousel backward/forward via `data-project-coverflow-previous`/`-next`, handled by the inline script's `showCoverflowCard()` function in `index.astro`.
Show My Pal JSON / Show Portfolio Workbench (button, dot indicators)	Jump directly to that project's card in the coverflow via `data-project-coverflow-dot` click handlers.
Search input (Name, description, or why…)	Filters the visible `[data-project-card]` elements client-side against each card's `data-search` attribute, toggling `hidden` and updating the empty-state/reset-filters UI (`data-filter-search` handler in `index.astro`'s inline script).

Page: /manage

The "Add a project" page, used to create a new project record — either committed to GitHub (hosted mode) or written to the local `src/content/projects/` folder (dev mode).



Controls

CONTROL	WHAT IT DOES

Contact card for Avishek Chandra / Share / + Add project (header buttons)	Same as on `/` — business card dialog, share dialog, and (already on this page) the add-project link, all from `BaseLayout.astro`.
Name input	Sets the new project's `name`; validated client-side (`required`, `maxlength=120`) and against `projectDraftSchema`/`projectMetadataSchema` server-side (or via GitHub commit in hosted mode).
Description textarea ("What does it do?")	Sets `description` (required, max 500 chars).
Why textarea ("What problem, curiosity, or need made you build it?")	Sets `why` (required, max 3000 chars).
Status select	Sets `status` to one of `idea | dev | review | delivered` (`projectStatuses` from `project-schema.ts`); per the project template, status can only be non-`idea` if a repository link exists.
Privacy select	Sets `privacy` to `private | public` (`privacyLevels`); hosted-created projects are always public per `RemoteCreate.astro`'s comment.
startDate input (date)	Sets the project's `startDate`; defaults to today's date (computed in the frontmatter of `manage.astro`).
Create project → (button, submit)	Submits the form; in local dev, the inline script (`manage.astro`) first calls `ensureDeviceUnlock()` (WebAuthn via `device-unlock-client.ts`), clears/shows field-level errors, and POSTs the draft to the local editor middleware (`local-editor.ts`'s `createProject` handler), which validates via `projectDraftSchema`, slugifies the name, checks for slug collisions, and writes a new `index.md` plus empty `artifacts/` folder. In hosted mode the equivalent handler in `RemoteCreate.astro`'s script commits a new `src/content/projects/<slug>/index.md` via the GitHub Contents API, guarding against duplicate repository links with a "This repository is already a project" dialog.

Page: /projects/my-pal-json

The detail page for the "My Pal JSON" project — a public project record showing its description, why-it-was-built rationale, metadata, and a full artifact carousel with inline editing controls.

PROJECT / 004 DEV

My Pal JSON

My Pal JSON is a web-based tool for testing APIs and analyzing JSON responses with features like dynamic header management, request/response viewing, and JSON structure visualization. It can generate code models in six programming languages (C#, Python, JavaScript, C++, Java, Go) and includes dark/light theme switching.

PRIVACY
Public
Ready to share.

STATUS	STARTED	UPDATED	REPOSITORY	PULL REQUEST
Dev	Oct 3, 2024	Sep 7, 2026	github.com/aviman1258/my-pal-json	—

LOCAL EDITOR

Manage this project

Changes are saved directly to this project's Markdown record.

✦ Edit project details

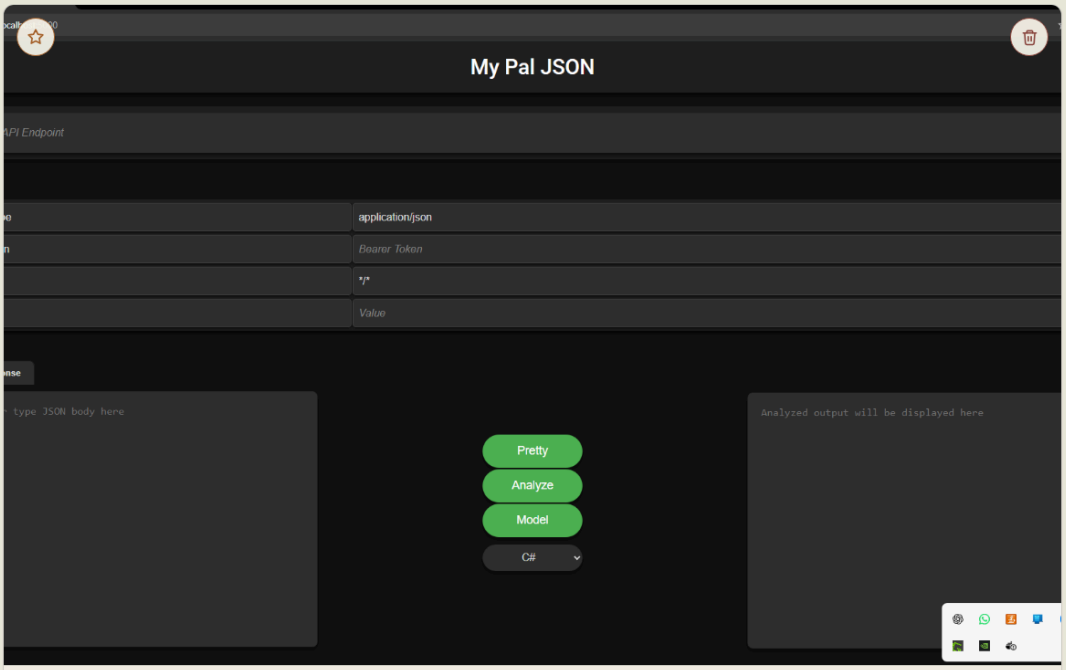
Why it was built

I built this project to eliminate the friction of context-switching betwe[illegible]orking with APIs—I found myself constantly moving JSON between my request client, a formatter, a structure inspector, and code generators. My Pal JSON consolidates all of these workflows into a single lightweight browser workspace, so I can send API requests, analyze responses, explore JSON structure, and generate code models without leaving the application. This unified approach saves time and keeps my focus on the actual API exploration rather than tool management.

Evidence from the work

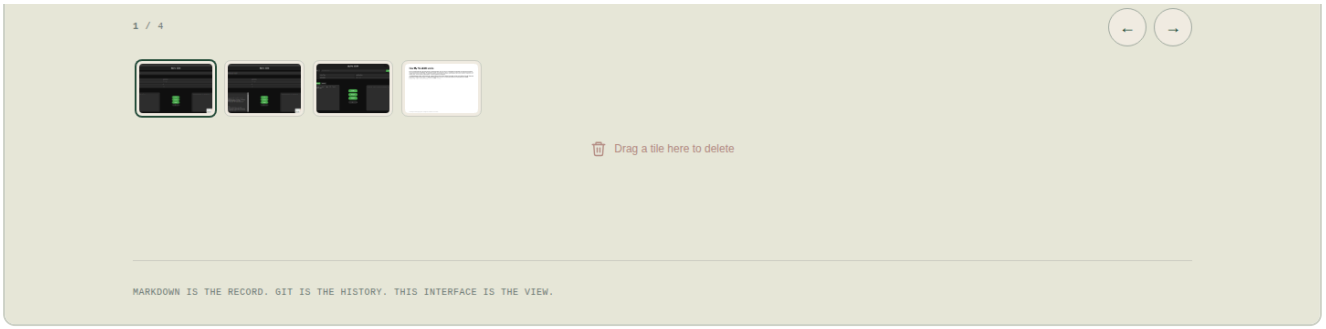
Images, documents, and recordings stored alongside this project. Drop files onto the carousel to add them, drag the tiles to reorder.

+ Add media



mypaljson-20260902055709.png
IMAGE

OPEN ORIGINAL ↗



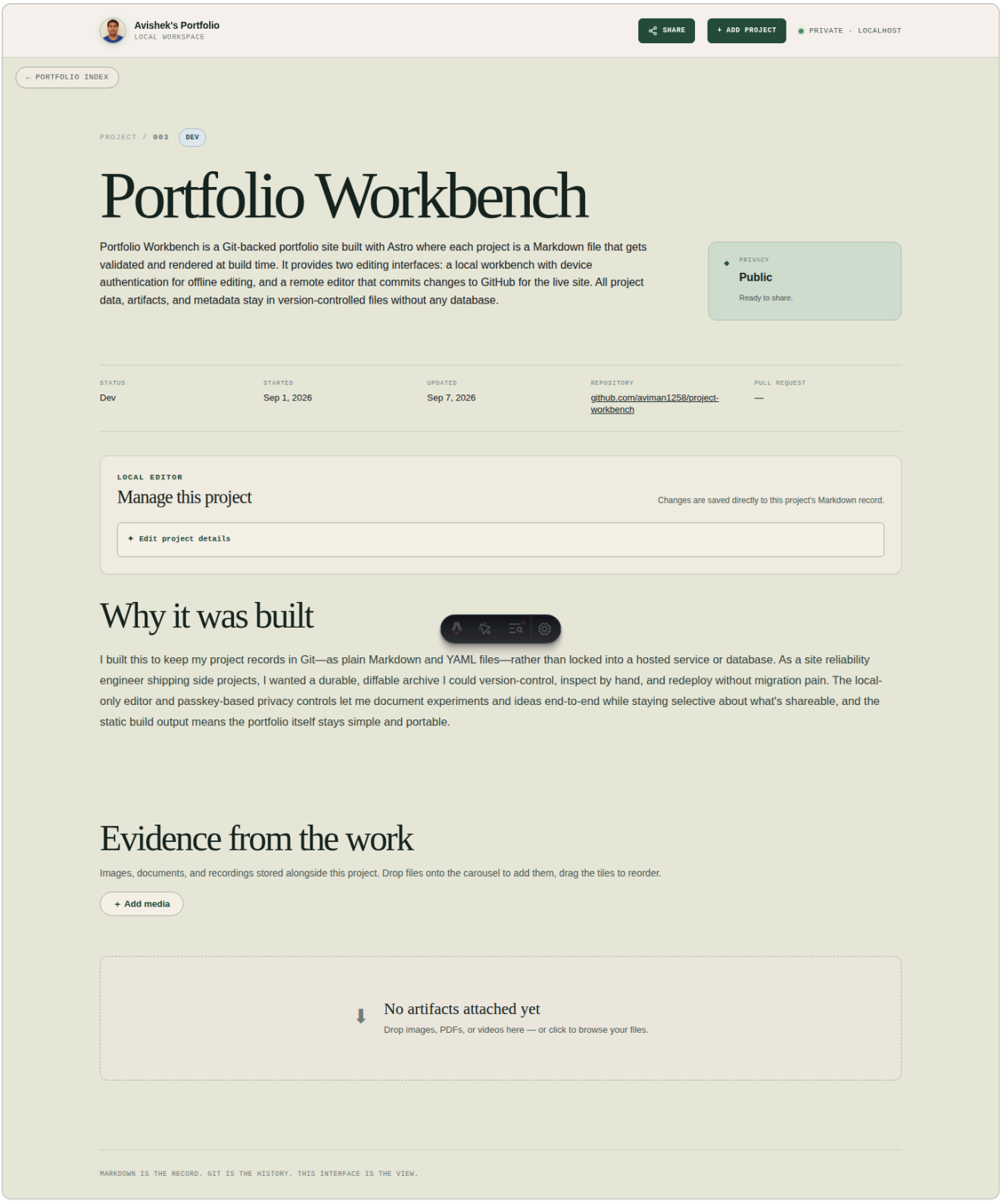
Controls

CONTROL	WHAT IT DOES
Contact card for Avishek Chandra / Share / + Add project (header)	Same as `/` — see BaseLayout.astro handlers.
github.com/aviman1258/my-pal-json (a)	External link to the linked GitHub repository (`data.repositoryUrl`, rendered in the metadata strip of `[slug].astro`).
✦ Edit project details (summary, details toggle)	Expands/collapses the inline project-editor form (`project-edit-details` in `[slug].astro`) containing name/description/why/status/privacy/startDate controls.
name / description / why / status / privacy / startDate fields	Edit the corresponding front-matter values; submitting via "Save changes" patches `index.md` (through `makeFrontmatterPatcher` in `project-editor.ts`, backed by `RemoteEditor`'s GitHub-commit `EditorBackend` or the local editor's file write) and bumps `updatedDate`.
Save changes (button, submit)	Commits/writes the edited front matter fields in one batch, updating `updatedDate` and refreshing the on-page display via `applyFieldDisplay()`.
+ Add media (button)	Opens a hidden file input (`data-remote-media-input` in `RemoteEditor.astro` or the local equivalent) to upload new artifact files; accepted files (with an extension, ≤25MB) are base64-encoded and committed to `artifacts/` via `putArtifact` in the `EditorBackend`.
Open <filename> (a)	Opens the artifact's raw file (served by `/project-artifacts/[project]/[...file].ts`) in a new tab (`target="_blank"`), generated by `carouselHtml()` in `artifact-carousel.ts`.

Feature <filename> (button)	Marks that artifact as the project's `featuredArtifact` in front matter (star toggle, `data-media-feature-trigger` wired in the carousel script) — the currently-featured artifact instead shows "<filename> is the featured artifact" and is visually disabled as a toggle target.
Delete <filename> (button)	Opens a confirm dialog and, on confirm, deletes that artifact file via `deleteFile`/local delete endpoint (`data-media-delete-trigger`).
Open original ↗ (a, figcaption)	Same as "Open <filename>" — direct link to the raw artifact file.
Show previous artifact / Show next artifact (button)	Step the artifact carousel backward/forward (`data-carousel-previous`/`-next`, wired in `artifact-carousel.ts`'s `wireCarousel`).
Show <filename> (button, thumbnail)	Jump the carousel directly to that artifact's slide (`data-carousel-thumbnail`).

Page: /projects/porfolio-workbench

The detail page for the "Portfolio Workbench" project itself (a public, self-documenting project record), currently with no artifacts uploaded.

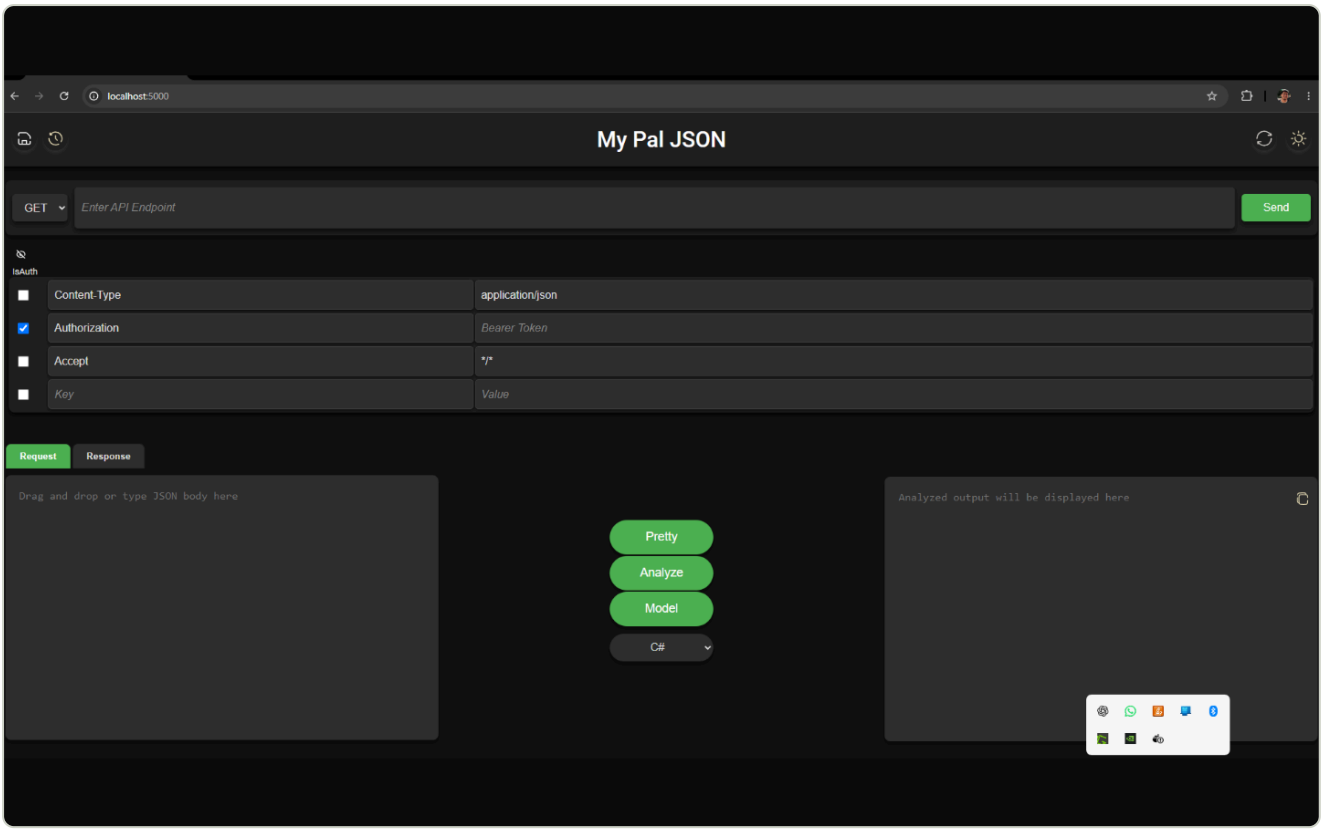


Controls

CONTROL	WHAT IT DOES
Contact card for Avishek Chandra / Share / + Add project (header)	Same as `/` — see BaseLayout.astro handlers.
github.com/aviman1258/project-workbench (a)	External link to the linked GitHub repository.
✦ Edit project details (summary)	Expands the inline editor form for name/description/why/status/privacy/startDate, same mechanism as the My Pal JSON page.
name / description / why / status / privacy / startDate fields + Save changes (button, submit)	Same patch-front-matter flow as described above, via `makeFrontmatterPatcher`/`EditorBackend`.
+ Add media (button)	Opens the file picker to upload the first artifact for this project (same handler as My Pal JSON's Add-media button).
⬇ No artifacts attached yet / drop-zone (button)	Acts as a clickable dropzone (`data-media-dropzone`) — clicking or dropping a file opens/uses the same upload flow as "+ Add media," per `carouselHtml()`'s empty-state markup in `artifact-carousel.ts`.

Page: /project-artifacts/my-pal-json/mypaljson-20260902055709.png

Direct static delivery of one of My Pal JSON's uploaded screenshot artifacts, served as a raw image response rather than an HTML page.

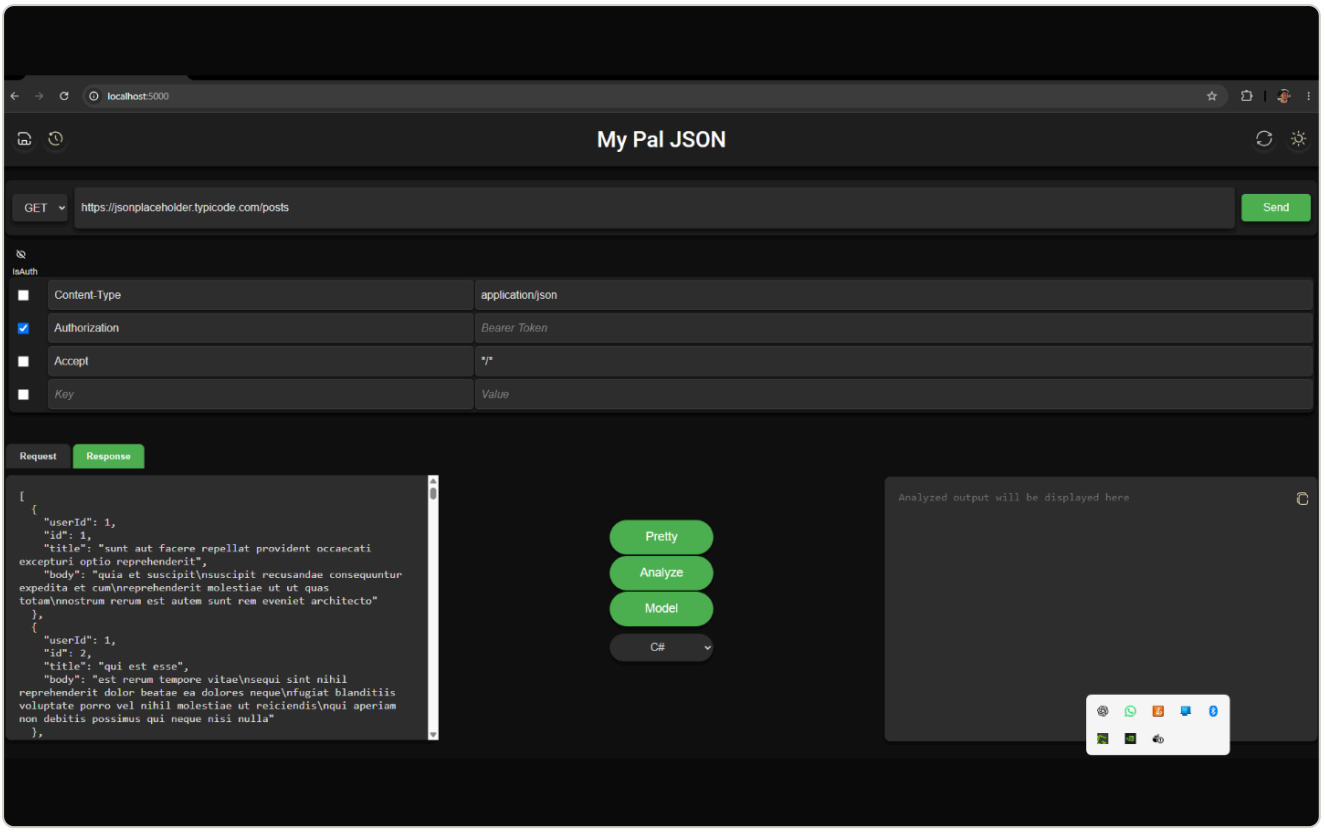


Controls

CONTROL	WHAT IT DOES
(none — this is a raw file response with no interactive controls)	

Page: /project-artifacts/my-pal-json/mypaljson2-20260902055735.png

Direct static delivery of My Pal JSON's second screenshot artifact (this one currently set as the project's `featuredArtifact`, per its front matter), served as a raw image.

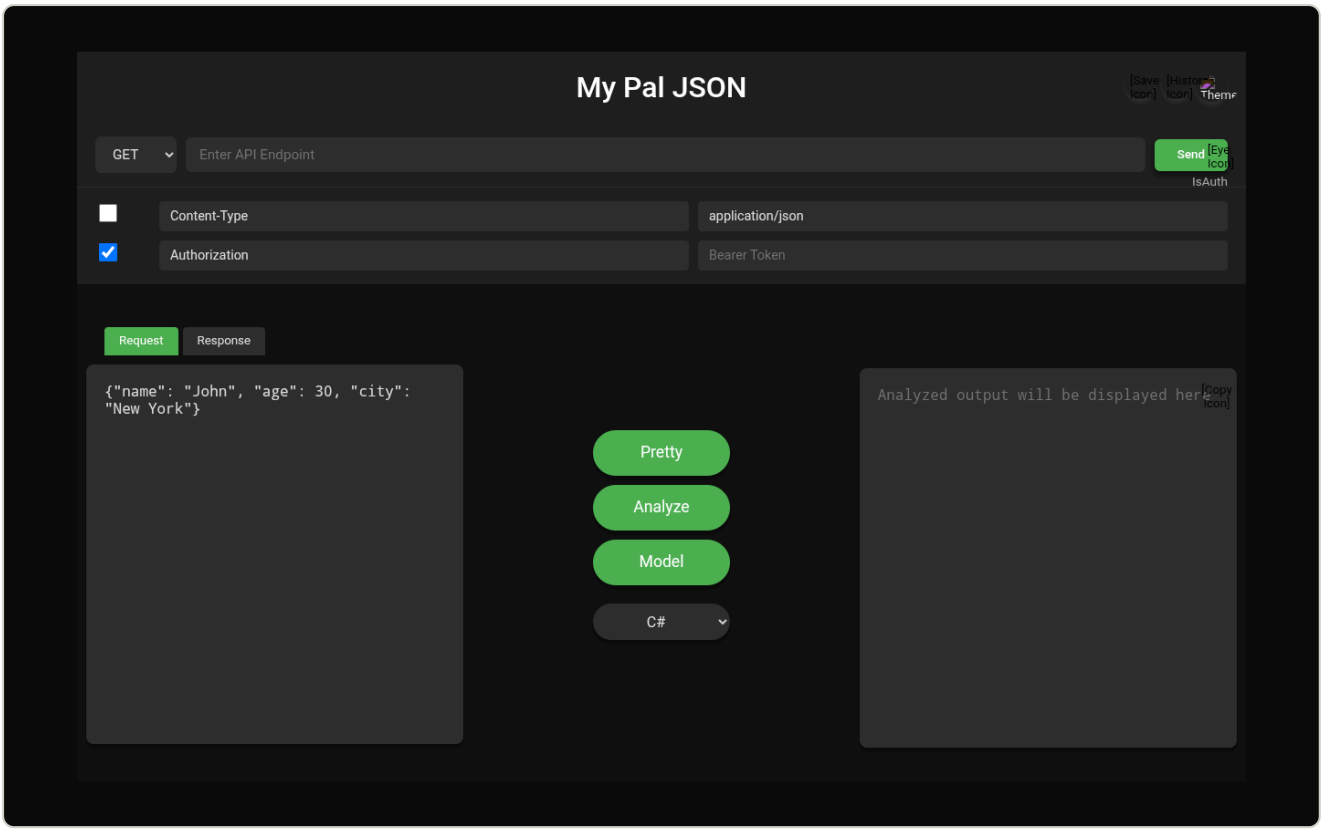


Controls

CONTROL	WHAT IT DOES
(none — this is a raw file response with no interactive controls)	

Page: /project-artifacts/my-pal-json/ui-main-json-api-tester.png

Direct static delivery of My Pal JSON's third screenshot artifact ("ui-main-json-api-tester.png"), served as a raw image.



Controls

CONTROL	WHAT IT DOES
(none — this is a raw file response with no interactive controls)	